

CSS

Mi a CSS?

Strukturált (pl. HTML, XML) dokumentumok megjelenítésének leírására szolgáló stíluslap nyelv. Szétválasztja a dokumentumok megjelenítési stílusát a dokumentumok tartalmától.

A CSS fejlesztése, CSS szintek

A W3C-n belül a CSS Munkacsoport fejleszti.

A szó hagyományos értelmében a CSS-nek nincsenek verziói, hanem szintjei vannak:

- CSS Level 1
- CSS Level 2
- CSS Level 3
 - Jelenleg is fejlesztés alatt áll.
 - Moduláris felépítésű:
 - Modulokra van bontva, ahol minden egyes modul a CSS egy részét definiálja.
 - A moduloknak is vannak szintjei: 1.szint: nincs megfelelője CSS2-ben, 3.szinttől: CSS2 létező lehetőségeit frissítő modulok.
 - A CSS modulok eltérő stabilitási szintűek.

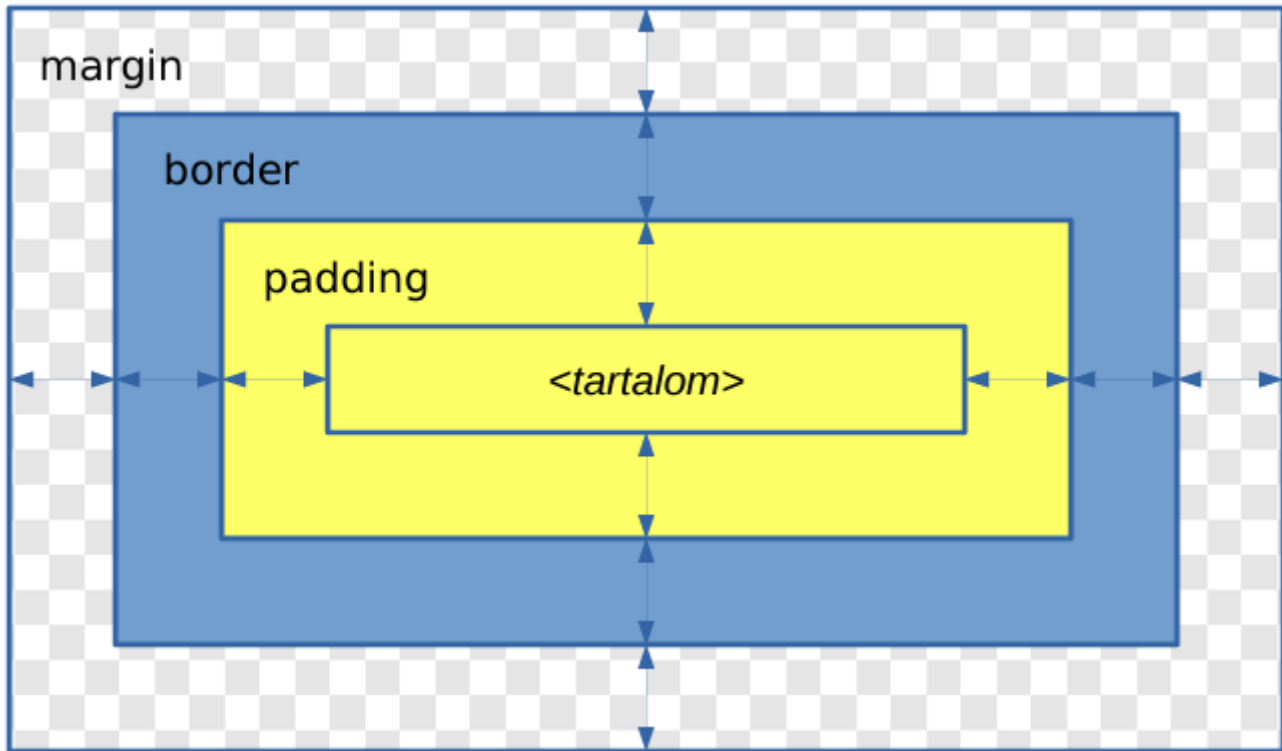
A CSS minden egyes szintje az előzőn alapul, annak definícióit f inomítja és új lehetőségeket vezet be.

CSS dobozmodell

A CSS egy fastruktúrájú dokumentumot kap, melyet egy rajzvásznon (például a képernyőn) jelenít meg egy olyan közbülső struktúrát, a dobozfát (box tree) előállítva, mely a megjelenített dokumentum formázási szerkezetét ábrázolja.

Minden egyes doboz a fában a dokumentum egy megfelelő elemét (vagy pszeudo-elemét) ábrázolja térben és/vagy időben a rajzvásznon.

A CSS minden egyes elemhez nulla vagy több dobozt generál az elem display tulajdonsága által meghatározott módon. Egy elem általában egyetlen dobozt generál.



Szintaktikai elemek

Karakterek

Az Unicode karakterkészlet használata.

Vezérlősorozatok

Unicode karakterek megadásához használhatunk \hhhhh formájú vezérlősorozatokat, ahol hhhhhh az Unicode karakter kódpontját ábrázoló legalább egy és legfeljebb 6 karakterből álló hexadecimális számjegysorozat.

- \9, \09, ..., \000009 a vízszintes tabulátor karaktert jelenti.
- \A9, \0A9, ..., \0000A9 a copyright szimbólumot (©) jelenti.

Speciális karakterek jelentésének elnyomásához használjuk a " karaktert. Például szükséges akkor, ha kiválasztóban pont karaktert tartalmazó elemnevet kell megadni.

- Például csak a given.name kiválasztóra illeszkedik a given.name nevű elem, a given.name kiválasztóra nem!

Megjegyzések

- A /* és */ határolók között lehet megadni megjegyzéseket.
- Példa: /* Style sheet for index.html */
- Tokeneken kívül bárhol megengedettek.
- Nem ágyazhatóak egymásba.

Deklarációs blokk

- '{' és '}' karakterek határolják, melyek között deklarációk egy listája kötelező.

- A deklarációk név:érték formájúak, ahol a tokenek előtt és után is megengedettek whitespace karakterek.
- A deklarációban név egy azonosító.
- A deklarációkat ';' karakterrel kell elválasztani.
- Az utolsó deklaráció után nem kötelező a ';' karakter.

at-szabályok

A stíluslap feldolgozását vezérlő speciális szabályok. Egy '@' karakterrel kezdődnek, melyet egy azonosító követ és ';' karakterrel vagy egy deklarációs blokkal végződnek.

- Példák: @charset, @import, @namespace, @media

Stílus szabályok

Egy kiválasztóból (vagy ';' karakterekkel elválasztott kiválasztókból) és egy az(oka)t követő deklarációs blokból állnak.

Tulajdonságok

A CSS által definiált paraméterek, melyek révén a dokumentumok megjelenítése vezérelhető. A tulajdonságoknak neve és értéke van.

Összevont tulajdonság (shorthand property): Olyan tulajdonság, mely több CSS tulajdonság értékének egyidejű beállítására szolgál. Például a margin tulajdonság a margin-top, margin-right, margin-bottom és margin-left tulajdonságok értékét állítja be.

Stílus eredet

Különböző eredetű stíluslapok állhatnak rendelkezésre dokumentumok megjelenítéséhez.

Felhasználói ágenstől származó

A felhasználói ágensnek biztosítanak alapértelmezett stíluslapot. Például olyan stílus szabály tartalmazása, mely az em HTML elem megjelenítéséhez kurzív betűtípust ír elő.

Felhasználótól származó

A felhasználó megadhat saját stíluslapot adott dokumentum megjelenítéséhez.

Szerzőtől származó

HTML esetén a *link* fejléc elemmel adható meg a dokumentumhoz külső stíluslap.

```
<link rel="stylesheet" href="style.css">
```

Beágyazott stílus a style fejléc elemmel:

```
<style>
  h1, h2, h3, h4, h5, h6 {
    font-variant: small-caps
  }
</style>
```

XML esetén az xml-stylesheet feldolgozási utasítással adható meg a dokumentumhoz külső stíluslap. A feldolgozási utasítás a dokumentum gyökéreleme előtt kell, hogy szerepeljen.

```
<?xml-stylesheet type="text/css" href="style.css"?>
```

"Fontos" deklaráció (!important)

Egy deklarációt követheti a '!' token és az **important** kulcsszó. Egy ilyen deklaráció felülír bármely más közönséges deklarációt.

```
color: red !important
```

Alapértelmezésben a szerzői stíluslap szabályok nagyobb precedenciával bírnak, mint a felhasználói stíluslap szabályok. A szerzői és a felhasználói stíluslapok is tartalmazhatnak !important deklarációkat, ilyenkor ezek precedenciája megfordul: a felhasználói !important szabályok felülírják a szerzői !important szabályokat.

```
table.chessboard > tbody > tr:nth-child(odd) > td:nth-of-type(odd){
  background-color: white;
}
table.chessboard > tbody > tr:nth-child(odd) > td:nth-of-type(even) {
  background-color: lightgray;
}
table.chessboard > tbody > tr > td:hover {
  background-color: salmon !important;
}
```

Ebben az esetben, ha nem lenne ott a !important az utolsó szabálynál, akkor az sose lenne aktív, mivel a felső két szabály specifikussága nagyobb, mint a harmadiké.

A kaszkád

Kaszkádolás: konfliktusfeloldási mechanizmus arra az esetre, amikor egy elem/tulajdonság kombinációhoz különböző deklarációk állítanak be értéket.

Több különböző deklaráció szolgáltathatja egy tulajdonság értékét egy elemhez. Ezek a deklarációk különböző eredetűek is lehetnek. A **kaszkád** az a folyamat, melynek során meghatározásra kerül, hogy a vonatkozó

deklarációk közül melyik határozza meg egy adott elem egy adott tulajdonságának értékét. A kaszkád az alábbi módon határozza meg egy adott elemhez egy adott tulajdonság kaszkádolt értékét:

- Meg kell határozni azokat a deklarációkat, melyek az adott elemhez az adott tulajdonság értékét szolgáltatják.
- Rendezzük a vonatkozó deklarációkat az eredetük szerint csökkenő "erőssorrendbe".
- Az azonos eredetű deklarációkat rendezzük specifikusság szerint csökkenő sorrendbe.
- A tulajdonság értékét a fenti sorrendben első deklaráció szolgáltatja.

Szabályok sorrendje

Lényeges lehet a szabályok sorrendje. Akkor számíthat a sorrend, ha egy elemre egynél több azonos specifikusságú stílus szabály vonatkozik. Ezek közül mindig a sorrendben utolsó a "legerősebb".

```
a:active {color: red} -> a = 0, b = 1, c = 1
a:hover {color: green} -> a = 0, b = 1, c = 1
a:visited {color: black} -> a = 0, b = 1, c = 1
a:link {color:blue} -> a = 0, b = 1, c = 1
```

Mivel itt a sorrendben utolsó a legerősebb, ezért az a művelet mikor rákattintunk / rávisszük az egeret sose fog teljesülni, mivel a másik két állapot egyike lesz látható. Ezért fontos a sorrend. A helyes sorrend:

```
a:visited {color: black} -> a = 0, b = 1, c = 1
a:link {color:blue} -> a = 0, b = 1, c = 1
a:hover {color: green} -> a = 0, b = 1, c = 1
a:active {color: red} -> a = 0, b = 1, c = 1
```

Öröklés

Öröklés, kezdőértékadás: abban az esetben alkalmazásra kerülő mechanizmusok, amikor egy elem/tulajdonság kombinációhoz egy deklaráció sem állít be értéket.

Az öröklés a tulajdonságértékek továbbadását jelenti a szülő elemektől a gyermek elemekhez. Bizonyos tulajdonságok öröklöttek, ami azt jelenti, hogy az értékük öröklés révén kerül meghatározásra, feltéve, hogy a kaszkád nem eredményez egy értéket.

A specifikáció minden egyes tulajdonsághoz meghatározza, hogy öröklött-e

- **Öröklött:** font-family, font-style, font-variant, font-weight, font-size, font tulajdonságok
- **Nem öröklött:** margin-top, margin-bottom, margin-right, margin-left, margin és width tulajdonságok

Egy tulajdonság kaszkádolt értékeként az inherit kulcsszó öröklést kényszeríti ki.

```
<span style="color: red">
  Az <em>erő</em> legyen veled!
</span>
```

Ebben az esetben, mivel az em HTML elemre nem vonatkozik color tulajdonság, így a span-tól örökli.

```
<style>
  a:visited, a:link {
    color: inherit;
  }
</style>

<p style="color: green">
  Click here: <a href="https://www.w3.org/">W3C</a>
</p>
```

Itt pedig az inherit kulcsszó miatt lesz a link zöld.

Hibakezelés

Ha a CSS-ben hiba fordul elő, akkor az elemző megpróbál abból úgy helyreállni, hogy csak a normál elemzéshez való visszatéréshez minimálisan szükséges tartalmat dobja el.

Minden egyes érvénytelen konstrukciót (deklarációt, stílus szabályt, at-szabályt) figyelmen kívül hagy a felhasználó ágens és folytatódik az elemzés.

- Ha egy stílus szabály kiválasztó listája egy érvénytelen kiválasztót tartalmaz, akkor a felhasználó ágens a teljes stílus szabály figyelmen kívül hagyja.
- Az érvénytelen, tehát figyelmen kívül hagyott deklarációk a DevTools-ban láthatók.

```
A
p {
  color: white
  background color: navy;
  font-family: sans-serif;
  unknown: 0;
  font-size: xxx;
}
stílus szabály gyakorlatilag ugyanaz, mint az alábbi:
p { font-family: sans-serif; }
```

Dobozok méretének meghatározása CSS-ben

box-sizing

A box-sizing CSS tulajdonság vezérli, hogyan kerül meghatározásra egy doboz szélessége és magassága. Azt határozza meg, hogy figyelembe kerül-e a keret és a belső margó (padding). A tulajdonság értéke *content-box* vagy *border-box*:

- **content-box:** a width és height CSS tulajdonságok a tartalom által elfoglalt (content area) méretét határozzák meg, azaz a keret és a belső margó nem kerül beszámításra.

```
div.box {  
  border: 10px solid black;  
  box-sizing: content-box;  
  height: 200px;  
  padding: 20px;  
  margin: 50px;  
  width: 300px;  
}
```

Szélesség: 360px, Magasság: 260px

- **border-box:** a width és height CSS tulajdonságok a szemmel is látható méretet határozzák meg, melybe beleszámít a keret és a belső margó is.

```
div.box {  
  border: 10px solid black;  
  box-sizing: border-box;  
  height: 200px;  
  padding: 20px;  
  margin: 50px;  
  width: 300px;  
}
```

Szélesség: 300px, Magasság: 200px

width és height

A width és height CSS tulajdonságok a dobozok **preferált szélességét és magasságát** határozzák meg.

min-width / min-height

A dobozok minimális szélességét/magasságát állítják be. Megakadályozzák, hogy a width/height tulajdonságok használt értéke kisebb legyen a min-width/min-height tulajdonságok előírt értékénél. Egy elem szélessége/magassága min-width/min-height értékére kerül beállításra, ha min-width/min-height értéke nagyobb mint max-width/max-height vagy width/height.

max-width / max-height

A dobozok maximális szélességét/magasságát állítják be. Megakadályozzák, hogy a width/height tulajdonságok használt értéke nagyobb legyen max-width/max-height előírt értékénél. A max-width/max-height tulajdonságok felülírják a width/height tulajdonságokat, azonban a min-width/min-height tulajdonságok felülírják a max-width/max-height tulajdonságokat.

CSS kiválasztók

A CSS kiválasztók mintaillesztésre szolgálnak. Meghatározzák, hogy egy szabály mely elemekre vonatkozik.

Kisbetű-nagybetű éréketlenség az ASCII tartományban (ekvivalensek az [a-z] és [A-Z] karakterek) az olyan részek kivételével, melyek nem a CSS hatálya alá esnek (elemnevek, attribútumnevek és -értékek).

Egyszerű kiválasztó

Egyetlen típus kiválasztó, általános kiválasztó, attribútum kiválasztó, osztály kiválasztó, ID-kiválasztó vagy pszeudo-osztály.

```
div - általános kiválasztó
[rel=stylesheet] - attribútum kiválasztó
.copyright - HTML osztály kiválasztó
#logo - ID kiválasztó
:lang(hu) - Pszeudo osztály kiválasztó
```

Kombinátor

Egy CSS-kiválasztó több egyszerű kiválasztót is tartalmazhat. Az egyszerű kiválasztók közé kombinátort kell beépíteni.

```
whitespace - leszármazott kombinátor
'>' - Gyermek kombinátor
'+' - Szomszéd testvér kombinátor
'~' - Általános kombinátor
```

Egyszerű kiválasztók sorozata

Egymást követő egyszerű kiválasztók, melyek között nincsenek kombinátorok. Típus kiválasztóval vagy általános kiválasztóval kezdődnek és nem tartalmaznak további típus kiválasztókat vagy általános kiválasztókat.

```
div
h2#status
link[rel=stylesheet][type=text/css]
p.copyright
*lang(hu) - *:univerzális kiválasztó
tr:nth-child(odd)
li:not(:last-child)
```

Kiválasztó

Egyszerű kiválasztók sorozatainak olyan sorozata, melyeket kombinátorok választanak el. Legalább egy sorozat alkotja. Pszeudo-elem at utolsó egyszerű kiválasztó sorozat végéhez adható hozzá, de legfeljebb csak egy!

```
p - 1 szekvencia bekezdés  
a img - olyan img elem ami leszármazottja a-nak  
h1 ~ table - ugyanahhoz a szülőhöz tartoznak  
div#main > h1 - h1 gyermeke a div#main-nek  
div > h1 + p  
q::before
```

Kombinátor

Leszármazott kombinátor

Egyszerű kiválasztók két sorozatát elválasztó whitespace.

Ha P és Q egyszerű kiválasztók két sorozata, akkor a P Q kiválasztóra a Q-ra illeszkedő olyan elemek illeszkednek, *melyek a P-re illeszkedő elem leszármazottai*.

```
thead th {  
    background-color: darkgrey;  
}
```

Gyermek kombinátor

Egyszerű kiválasztók két sorozatát elválasztó '>' karakter.

Ha P és Q egyszerű kiválasztók két sorozata, akkor a P > Q kiválasztóra a Q-ra illeszkedő olyan elemek illeszkednek, *melyek a P-re illeszkedő elemek gyermekei*.

```
nav > div {  
    display: inline;  
}  
p > img:only-child {  
    margin-left: 0;  
}
```

Szomszéd testvér kombinátor

Egyszerű kiválasztók két sorozatát elválasztó '+' karakter.

Ha P és Q egyszerű kiválasztók két sorozata, akkor a P + Q kiválasztóra a Q-ra illeszkedő olyan elemek illeszkednek, *melyek a P-re illeszkedő elemet követnek közvetlenül a dokumentumban*.

- Az illeszkedő elemeknek ugyanaz kell, hogy legyen a szülője.
- Köztük megengedettek olyan konstrukciók, melyek nem elemek, például szöveg és megjegyzések, ezek figyelmen kívül hagyása.

```
h1 + p {  
  text-indent: 0;  
}
```

Általános testvér kombinátor (CSS3)

Egyszerű kiválasztók két sorozatát elválasztó '-' karakter.

Ha P és Q egyszerű kiválasztók két sorozata, akkor a $P \sim Q$ kiválasztóra a Q-ra illeszkedő olyan elemek illeszkednek, melyek a P-re illeszkedő elemet követnek (nem felétlenül) közvetlenül a dokumentumban.

- Az illeszkedő elemeknek ugyanaz kell, hogy legyen a szülője.

```
img ~ span {  
  color: red;  
}
```

Kiválasztók típusai

Típus kiválasztó

Egy CSS minősített név, a gyakorlatban tipikusan egy azonosító. A megfelelő nevű elemek illeszkednek rá.

```
p {color: red } - A p elemek színe legyen piros  
a { text-decoration: none} - Ha az elem link, akkor nem lesz aláhúzva
```

Általános kiválasztó

Általános kiválasztónak nevezzük a * formájú kiválasztót. Minden elem illeszkedik rá. Elhagyható olyan egyszerű kiválasztóból, mely további komponenseket is tartalmaz.

Ekvivalens kiválasztók

```
*#nav = #nav  
*.important = .important  
*[title] = [title]
```

Nem ajánlott elhagyni, az olvashatóság végett

```
div *:first-child = div :first-child  
div *: first-child != div: first-child
```

Attribútum kiválasztók

Az attribútum kiválasztókban érték azonosító vagy sztring. Tehát például a következők ekvivalensek:

```
[dir=rtl]
[dir='rtl']
[dir="rtl"]
```

Fajtai [att]: az att attribútummal rendelkező elemek illeszkednek rá

[att=érték]: olyan elemek illeszkednek rá, melyek att attribútumának értéke pontosan érték

```
style[type=italic] {
  font-style: italic;
}
style[type=bold] {
  font-weight: bold;
}
style[type=normal] {
  font-style: normal;
  font-weight: normal;
}
```

Osztály kiválasztó

HTML dokumentumoknál a [class ~= érték] attribútum kiválasztó helyett használható az ekvivalens .érték kiválasztó.

div: Általános célú blokk, amit arra használnak, hogy egymástól elkülönítsék a tartalmat

centered: középre igazítható

margin-left:auto; - margin-right: auto; vízszintesen középre igazítás

```
div.centered {
  margin-left: auto;
  margin-right: auto;
}
```

Az important azonosítójú elemek lesznek azok amiket elsődlegesként kezelnek. color: red; piros szín beállítása
text-decoration: underline; a szöveg aláhúzott

```
.important {
  color: red;
  text-decoration: underline;
}
```

ID-kiválasztó

#azonosító formájú kiválasztó, az adott azonosítójú elem illeszkedik rá.

Az azonosítót egy ID típusú attribútum kell, hogy szolgáltatssa a dokumentumban. A dokumentum nyelvétől függ, hogy mi ennek az attribútumnak a neve, például a HTML-ben id.

```
div#main {  
    width: 50%;  
    margin-left: auto;  
    margin-right: auto;  
}
```

```
#footer {  
    text-align: center;  
}
```

Pszeudo-osztályok

:azonosító vagy :azonosító(érték) formájú kiválasztó

Jellemzői:

- **Kisbetű-nagybetű érzéketlen**, általában kisbetű
- **Olyan kiválasztást tesznek lehetővé, mely a dokumentumon kívül információkon alapul vagy nem fejezhető ki a többi egyszerű kiválasztóval.**
- **Bizonyos pszeudo-osztályok egymást kölcsönösen kizáróak** (pl. :link és :visited)
- Bizonyos pszeudo-osztályok **dinamikusak**: ezek olyan pszeudo-osztályok, melyeket egy elem megszerezhet vagy elveszíthet, miközben a felhasználó interakcióban van a dokumentummal.
 - **A dinamikus pszeudo-osztályok két fajtája**: link pszeudo-osztályok és felhasználói akció pszeudo-osztályok.

Dinamikus pszeudo-osztályok

Link pszeudo-osztályok

:link: a felhasználó által még nem meglátogatott hiperhivatkozásokra vonatkozik.

```
a:link {  
    color: indigo;  
    text-decoration: none;  
}
```

:visited: a felhasználó által meglátogatott hiperhivatkozásokra vonatkozik

```
a:visited {  
  color: gray;  
  text-decoration: line-through;  
}
```

Felhasználói akció pszeudo-osztályok

:hover: a felhasználó által mutatóeszkővel kijelölt, de nem feltétlenül aktivált elemre vonatkozik.

```
a:hover {  
  border-width: medium;  
  border-style: solid;  
}
```

:active: a felhasználó által mutatóeszkővel aktivált elemre vonatkozik. Például az egérgomb lenyomása és felengedése között hatásos.

```
a:active {  
  font-weight: bolder;  
}
```

:focus: a fókuszt birtokló elemre vonatkozik.

További pszeudo-osztályok

A :target pszeudo-osztály

Ha a dokumentum URI-ja tartalmaz erőforrásrész-azonosítót, akkor az erőforrásrész-azonosítónak megfelelő elem kiválasztása. Ha például `index.html#copyright` a dokumentum URI-ja, akkor a `div:target` kiválasztó a `copyright` azonosítójú `div` elemet választja ki.

A :lang(C) pszeudo-osztály:

A C nyelvű szöveget tartalmazó elemek illeszkednek rá, ahol C egy CSS azonosító (nyelvkód). Például:

:lang(en)

XML dokumentumokban a nyelvet az `xml:lang` attribútum határozza meg, HTML dokumentumokban a nyelv megadható a `lang` és az `xml:lang` attribútummal is. A *lang* és *xml:lang* attribútum nem közvetlenül az elemen kell, hogy megjelenjen.

```
A :lang(C) pszeudo-osztály használatakor magyar nyelvű idézet esetén:  
q:lang(hu){  
  quotes: "" "" ">>" "<<";  
}
```

ahol a `q`, egy rövid idézet ami nem tör új sort. A `lang(hu)` jelöli a magyar nyelvet. A `quotes` mondja meg, hogy milyen határolók között legyen. A `"<<"` és párja jelöli azt, hogy mire cserélje az idézőjeleket ha alpból van a szövegben.

:is() pszeudo-osztály

`:is(X)` formában adható meg, ahol az argumentum egy kiválasztó lista. Matches-any pseudo-class néven is ismert. Minden olyan elem illeszkedik rá, mely illeszkedik a lista argumentum valamely kiválasztójára. Hosszú kiválasztó listák tömörebb alakban való ábrázolásához hasznos.

A pszeduo-elemek nem érvényesek az `:is()`-en belül. Az `:is()` kiválasztó specifikussága az argumentum lista legnagyobb specifikusságú kiválasztójának specifikussága.

`:is()` pszeudo-osztály: ekvivalens például az alábbi két stíluslap szabály:

```
div:is(.note, .warning, .hint)::before{
    /*...*/
}

div.note::before,
div.warning::before,
div.hint::before{
    /*...*/
}
```

A negáció pszeudo-osztály

`:not(X)` formában adható meg, ahol az argumentum egy olyan egyszerű kiválasztó, melyben nem megengedett a negáció pszeudo-osztály. Azok az elemek illeszkednek rá, melyek nem illeszkednek az argumentumra. Pl.: `:not(:important)`, `:not([title])`, `li:not(:last-child)`

Szerkezeti pszeudo-osztályok

Amikor megállapításra kerül, egy elem helye a testvéreinek listájában, akkor csak az elemeket kell a listában figyelembe venni. Nem kell például figyelembe venni a szöveget, a megjegyzéseket és a feldolgozás utáni utasításokat. A listában az elemek számozása egytől egyig történik.

- **:root**: a dokumentum gyökérelemét választja ki
- **:only-child**: olyan elemeket választ ki, melyek szülőjének nincs más elemgyermeke
- **:only-of-type**: olyan elemeket választ ki, melyek szülőjének nincs más az elemmel megegyező kifejtett nevű elemgyermeke
- **:empty**: a szöveget és elemeket nem tartalmazó elemeket választja ki, melyek azonban tartalmazhatnak megjegyzéseket és feldolgozási utasításokat.
- **:nth-child(an+b)**: az olyan elemeket választja ki, melyeknek $an+b-1$ megelőző elemtestvére van valamely nem negatív n -re.
- **:nth-last-child(an+b)**: az olyan elemeket választja ki, melyeknek $an+b-1$ következő elemtestvére van valamely nem negatív n -re.

- **:nth-of-type(an+b)**: az olyan elemeket választja ki, melyeknek valamely nem negatív n-re $an+b-1$ olyan megelőző elemtestvére van, melyek kifejtett neve megegyezik az elem kifejtett nevével.
- **:nth-last-of-type(an+b)**: az olyan elemeket választja ki, melyeknek valamely nem negatív n-re $an+b-1$ olyan következő elemtestvére van, melyek kifejtett neve megegyezik az elem kifejtett nevével.
- **:first-child**: azt jelenti, mint `:nth-child(1)`
- **:last-child**: azt jelenti, mint `:nth-last-child(1)`
- **:first-of-type**: azt jelenti, mint `:nth-of-type(1)`
- **:last** Argumentumokat meg lehet adni röviden:
 - $an+0 \rightarrow an$
 - $0n+b \rightarrow b$
 - Negatív b esetén $an-b$ módon kell megadni az argumentumot, nem megengedett $an+-b$
 - A $2n$ argumentum megadható *even* módon, a $2n+1$ argumentum pedig *odd* módon.

Pszeudo-elemek

Lehetővé teszik a dokumentumok olyan részeinek kiválasztását, melyek más módon nem hozzáférhetők. Minden kiválasztóban legfeljebb egy pszeudo-elem megengedett. Az alábbi pszeudo-elemek állnak rendelkezésre:

- `::first-line`
- `::first-letter`
- `::before`, `::after` A pszeudo-osztályoktól való megkülönböztetés miatt ':' helyett '::' karaktereket használ a pszeudo-elemekhez a CSS3.

`::first-line`

Egy elem első formázott sorát ábrázolja.

```
p::first-line{
    text-decoration: underline
}
```

`::first-letter`

Egy elem első betű vagy számjegy karakterét ábrázolja, ha azt ben előzi meg más tartalom, például kép. Vonatkozik az első betű vagy számjegy karaktert megelőző vagy követő írásjelekre is.

```
p::first-letter {
    font-size: 2em;
    font-weight: bold;
}
```

`::before`, `::after`

A `::before` és `::after` lehetővé teszik egy elem tartalmához generált tartalom hozzáadását.

```
div.proof::before{
  content: "Proof: ";
  font-weight: bold;
}

div.proof::after{
  content: "\220E" /* End of proof */
}
```

Specifikusság, a specifikusság meghatározása

Kiválasztókhoz és deklarációkhoz specifikusság meghatározása. A specifikusság egy háromelemű (a, b, c) vektor, ahol a, b és c nemnegatív egészek. A vektorok rendezése lexikografikusan történik.

Specifikusság meghatározása:

- **a:** a kiválasztóban előforduló ID-kiválasztók száma
- **b:** a kiválasztóban előforduló attribútum kiválasztók és pszeudo-osztályok száma
 - A negáció pszeudo-osztályt figyelmen kívül kell hagyni, azonban az argumentumát nem!
- **c:** a kiválasztóban előforduló típus kiválasztók és pszeudo-elemek száma.

Deklarációk specifikussága: Egy deklaráció specifikussága megegyezik a tartalmazó szabály kiválasztójának specifikusságával. Szabályhoz nem tartozó (HTML-ben a style attribútum értékeként adott) deklaráció specifikussága nagyobb minden kiválasztóénál.

Kiválasztó	ID kiválasztó szama	attribútum kiválasztó pseudo-szama	típus kiválasztó pseudo-szama
*	0	0	0
div	0	0	1
p::first-letter	0	0	2
a img	0	0	2
h1 + p	0	0	2
div[class = nav]	0	1	1
div.nav	0	1	1
#main *:lang(en)	1	1	0